

Experiment 1: Object Detection and Recognition

Student: _____
Student ID: _____

August 2026

Abstract

This experiment develops and deploys a four-class desktop-object detector for mouse, laptop, cup, and phone. Images were collected from videos, annotated in ISAT, converted to YOLO format, and used to fine-tune YOLOv8n. The final checkpoint was executed on a Jetson Orin with a USB camera. On a correlated temporal-frame diagnostic set, the detector matched 103 of 104 annotated objects, giving 100.00% precision and 99.04% recall. A representative Jetson frame displayed simultaneous detections and 15.0 FPS. These results demonstrate a working prototype, while the required independent 20-object accuracy test, stable averaged FPS log, and end-to-end ROS2 runtime record must still be completed before the results can be claimed as final course acceptance.

1 Objective and Acceptance Criteria

The objectives were to (1) detect at least two types of desktop objects, (2) train a detector using self-collected and annotated data, (3) run the detector on Jetson, (4) display class, bounding box, confidence, and frame rate in real time, and (5) publish detection results through ROS2. The course acceptance criteria require at least two simultaneously recognizable classes, testing on at least 20 objects with accuracy no lower than 80%, Jetson speed of at least 5 FPS, and preservation of result videos and typical error cases.

2 Hardware and Software

Table 1: Experimental environment

Item	Configuration
Training computer	Windows 11; NVIDIA GPU and CUDA details should be filled from the training machine.
Embedded computer	Jetson Orin, aarch64, Ubuntu 20.04.6, L4T R35.6.4 / JetPack 5.1.6.
GPU software	CUDA 11.4; NVIDIA PyTorch 2.0.0a0; torchvision 0.15.1 with working CUDA NMS.
Python and framework	Python 3.8 on Jetson; Ultralytics 8.4.22; YOLOv8n.
Camera	USB camera at 640×480 input; exact model: _____.
ROS2	‘roslpy’, ‘cv_bridge’, ‘sensor_msgs’, and ‘vision_msgs’; distribution: _____.

3 Dataset and Annotation

The dataset contains 595 images from self-recorded videos. The four classes are **mouse**, **laptop**, **cup**, and **phone**. ISAT polygon annotations were converted to YOLO bounding boxes. The conversion script recomputes each box from the polygon, validates normalized coordinates, removes near-duplicate boxes, and records provenance in a manifest.

Table 2: Dataset split and object counts

Split	Images	Objects	Objects per class (mouse/laptop/cup/phone)
Train	421	506	119/85/212/90
Validation	87	104	28/17/42/17
Test diagnostic	87	104	28/17/42/17

The split uses contiguous temporal blocks within each of 19 source groups. This prevents random frame shuffling, but the same physical source groups occur in all three splits. Therefore, neighboring-frame correlation remains and the diagnostic set is not an independent physical-object benchmark.

4 Method

The COCO-pretrained YOLOv8n model was fine-tuned as a four-class detector. Training used 100 epochs, 640-pixel input images, batch size 16, GPU device 0, zero data-loader workers, deterministic seed 42, automatic optimizer selection, standard YOLO data augmentation, and early-stopping patience of 25. Training used only the train and validation splits; the diagnostic test split was not used for optimization.

Listing 1: Reproduction command

```
python train_yolo.py --epochs 100 --batch 16 --device 0 --workers 0
```

5 Training and Frame-Level Diagnostic Results

The best recorded validation mAP@0.5:0.95 was 0.98571 at epoch 76. At epoch 100, precision was 0.99417, recall was 0.99405, mAP@0.5 was 0.99104, and mAP@0.5:0.95 was 0.98078. The complete curves and confusion matrices are stored in `training/yolov8n_4class_finetune/`.

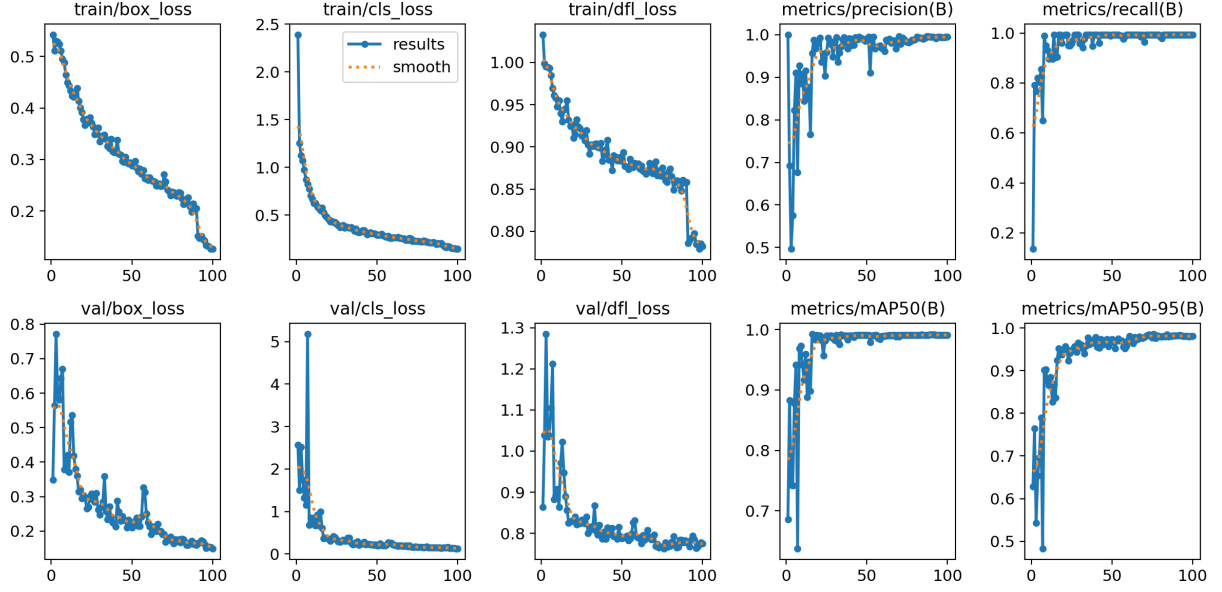


Figure 1: Training losses and validation metrics over 100 epochs. The curves show convergence and late-stage stabilization. Because only one run is available, run-to-run variation cannot be estimated.

At confidence threshold 0.25 and matching IoU threshold 0.50, 103 of 104 annotated objects were matched. There was one false negative and no false positives.

Table 3: Frame-level diagnostic by class

Class	Ground truth	Correct	FN	Recall
Mouse	28	28	0	100.00%
Laptop	17	17	0	100.00%
Cup	42	41	1	97.62%
Phone	17	17	0	100.00%
Total	104	103	1	99.04%

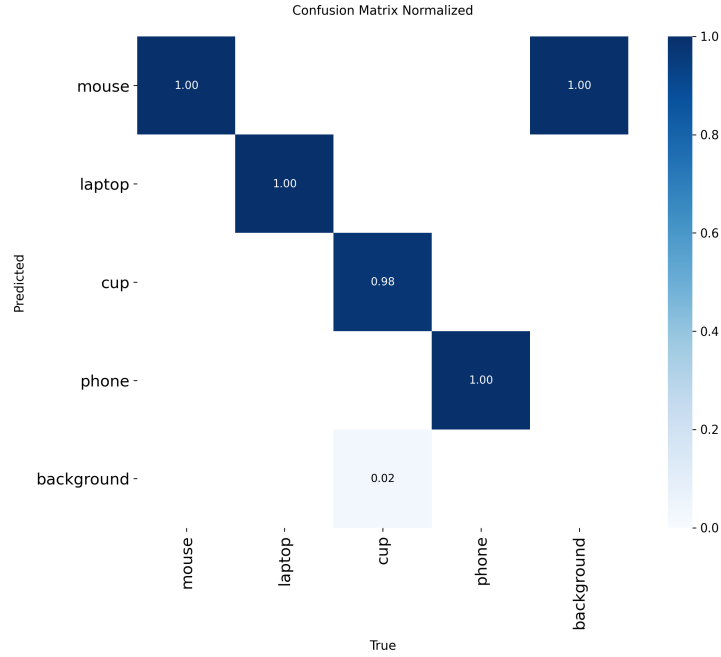


Figure 2: Normalized validation confusion matrix. The dominant diagonal indicates that class confusion was limited on the validation frames.

The 99.04% value is a frame-level diagnostic result only. It must not be reported as the final real-world accuracy because temporal frames from related source groups occur in both training and testing.

6 Jetson Deployment and Real-Time Detection

The checkpoint `weights/best.pt` was copied to Jetson and loaded by `realtime_detect.py`. The program uses V4L2 for a USB camera, performs GPU inference, draws bounding boxes and confidence scores, counts objects by class, calculates a sliding-window FPS, and optionally writes an annotated MP4 file.

Listing 2: Jetson real-time command

```
python realtime_detect.py --weights ~/robotics_exp1/weights/best.pt \\  
  --source 0 --imgsz 320 --confidence 0.35 --device 0 \\  
  --camera-width 640 --camera-height 480 --camera-fps 30 \\  
  --output ~/robotics_exp1/results/demo.mp4
```

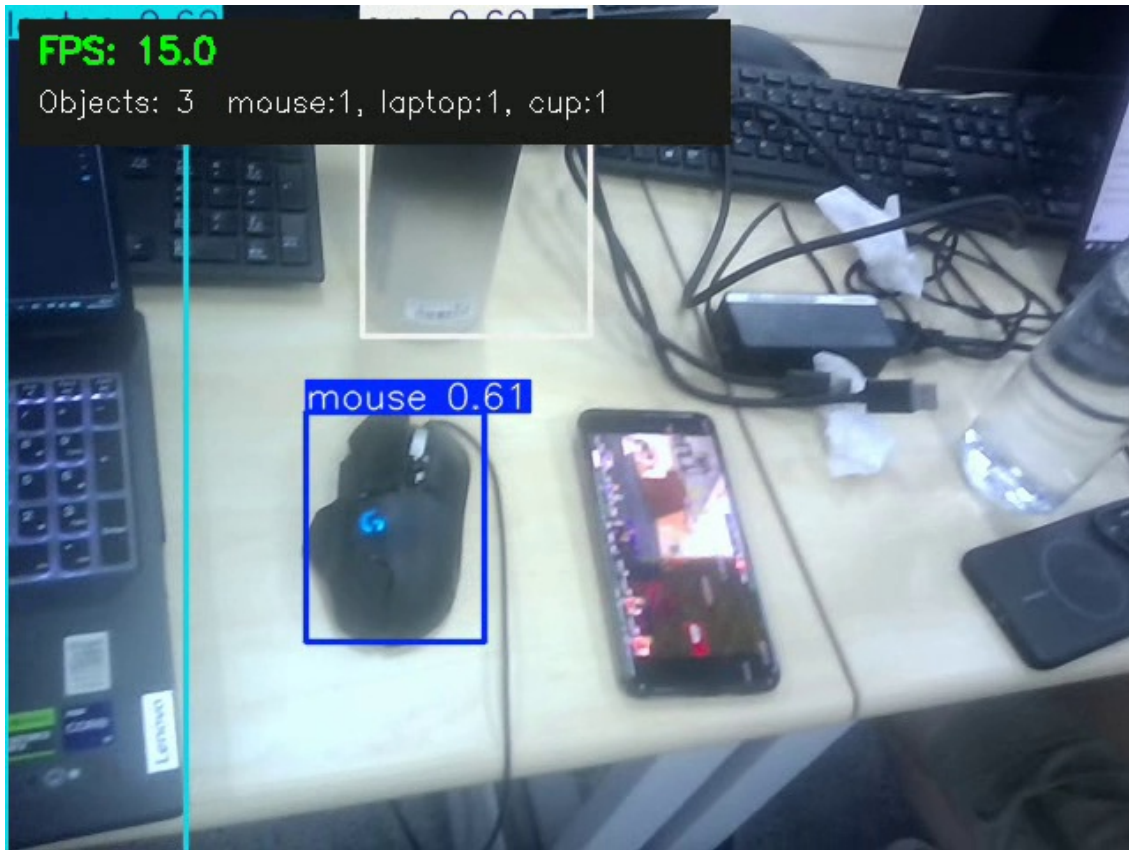


Figure 3: Representative Jetson demo frame. The overlay shows simultaneous mouse, laptop, and cup detections, confidence scores, object counts, and a displayed FPS of 15.0. This is an observed sliding-window value, not a saved time-averaged benchmark.

The saved demonstration video is `results/jetson_demo.mp4`. The final report should add the measured mean and minimum FPS over a continuous 30–60 second run. Required field: **mean FPS** = _____; **minimum FPS** = _____; **pass (mean $\geq$ 5 FPS)**: _____.

7 ROS2 Publication

The node `ros2_detector_node.py` subscribes to `/camera/image_raw` as `sensor_msgs/Image`. It converts images with `cv_bridge`, performs YOLO inference, and publishes `vision_msgs/Detection2DArray` on `/detections`. Each detection contains a pixel-coordinate center, width, height, class name, and confidence score.

Listing 3: ROS2 verification command

```
python ros2_detector_node.py --weights ~/robotics_exp1/weights/best.pt
\\
--image-topic /camera/image_raw --output-topic /detections --device 0
ros2 topic echo /detections --once
```

The implementation is present, but an end-to-end runtime capture has not yet been archived. Insert the terminal screenshot or bag/topic record here: **ROS2 evidence:**

8 Independent 20-Object Test

The course accuracy requirement must be measured on at least 20 distinct physical objects that were not merely adjacent video frames. Record one row per object in `results/independent_test_template.csv`.

Table 4: Independent acceptance test (to be completed)

Class	Tested	Correct	Accuracy
Mouse	_____	_____	_____
Laptop	_____	_____	_____
Cup	_____	_____	_____
Phone	_____	_____	_____
Total	_____	_____	_____

Accuracy is calculated as $100 \times$ correctly recognized objects divided by tested objects. For exactly 20 objects, at least 16 correct predictions are needed to meet 80%.

9 Typical Error Cases and Discussion

The saved error image `results/yolov8n_4class_finetune_test/errors/cup_cup4_000086.jpg` records the only missed object in the frame diagnostic. Cups can be difficult because reflective surfaces, changing viewpoints, partial occlusion, and visually similar backgrounds weaken appearance cues. Practical improvements include collecting more viewpoints and lighting conditions, adding background diversity, balancing source groups, checking annotation consistency, and tuning the confidence threshold on an independent validation set.

10 Conclusion

This experiment produced a deployable YOLOv8n detector for four desktop-object classes and demonstrated Jetson-side real-time visualization with an observed 15.0 FPS overlay. The saved frame-level diagnostic reached 103/104 matched objects (99.04% recall/object accuracy), with one cup miss. The code also implements ROS2 publication and result-video saving. The strongest supported conclusion is that the prototype is operational; final course acceptance must be confirmed by inserting the independent 20-object accuracy table, a stable averaged FPS measurement, and ROS2 topic evidence. No claim of final independent accuracy is made before those measurements are completed.

Reproducibility Index

- Dataset configuration: `dataset/yolo/data.yaml`; conversion summary: `dataset/yolo/conversion_summary.json`.
- Final checkpoint: `weights/best.pt`; SHA-256: 595C8FD32BE2A733BF0845A5FC89847DAD425A1FDEC249C0375E0DEA9B9CFBA5.
- Training metrics: `training/yolov8n_4class_finetune/results.csv` and plots.

- Evaluation: `results/yolov8n_4class_finetune_test/summary.json` and `per_image.csv`.
- Deployment and ROS2 source: `jetson_setup/deploy_to_jetson.ps1`, `realtime_detect.py`, and `ros2_detector_node.py`.
- Demonstration video: `results/jetson_demo.mp4`.